

UROP Project Report

UROP - Report

Visualisation of the Code Review Process

Daniel Bee

A report submitted in part fulfilment of the scholarship of

BSc (Hons) in Computer Science

Supervisor: DongGyun Han



Department of Computer Science
Royal Holloway, University of London

June 27, 2024

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: TBC

Student Name: Daniel Bee

Date of Submission: 28th June 2024

Signature: Daniel Bee

Table of Contents

Abstract	3
Project Specification	4
1 Introduction	5
1.1 Project Prototype Screenshot	5
2 Crawler	6
3 Database	7
3.1 Database Solution	7
3.2 Database Schema	7
3.3 Migration	8
4 Frontend	9
4.1 JavaScript vs TypeScript	9
4.2 ESLint & Prettier	9
4.3 Developer relationship algorithm	10
4.4 Applying D3.js	11
5 Backend	12
6 Reflection	13

Abstract

This report covers the design and development cycle of the "Visualising Interactions Between Developers During Code Review" UROP project. This includes the thought processes throughout the project, as well as the many challenges that were faced along the way and how these were dealt with. There are four primary milestones that were allocated and scheduled for this project: Crawler, Database, Frontend, and Backend. Firstly, the Crawler milestone was where pull request information from the TypeScript GitHub repository from Microsoft was fetched via the GitHub GraphQL API. Secondly, the Database milestone was the process of designing an appropriate relational database schema to hold this newly retrieved pull request information for easier analysis. Thirdly, the Frontend milestone was where mock data was inputted into a forced-directed graph powered by D3.js and React to replace this mock data with the pull request information from the relational database later on. Fourthly, and finally, the Backend milestone was the section of the project where everything came together. During this milestone, the frontend graph was linked to the backend API endpoint to allow data to be transferred to a callback from axios, a promise-based HTTP client for browsers and node.js.

Project Specification

The UROP project aims to create a full stack, web-based system that allows users to view the relationships between developers during the Code Review process. Python is used to code the initial crawling stage of the project, making use of the requests pip library to make calls to the GitHub GraphQL API endpoint with an appropriate template string. Python is also used in the migration stage of the project to move the newly acquired pull request information from individual JSON files to an efficient relational database, using the mysql-connector-python pip library to perform this. The frontend handles the visualising of the data itself, and is primarily coded in TypeScript, whilst the backend, coded in Java, makes queries to a local relational database to consummate the visualisation. Axios comes in handy for the frontend stage of the project to make the call to the backend API endpoint, and allows users to easily add parameters to a callback. D3.js is an invaluable tool in deciphering the response Axios receives from that callback, and subsequently produces the final visualisation the user sees. Lastly, Spring Boot was vital in allowing easy development of the REST API building blocks that were essential to the development of the project. This is particularly clear when Spring Boot becomes linked to the relational database containing the pull request information since Spring Boot makes it incredibly simple to make queries or to mutate the data inside the database tables, if necessary.

Chapter 1: Introduction

Visualising complex and congested data is a challenge, no matter how well-versed a developer might be in APIs. Therefore, to help aid in this process, several APIs and frameworks can be used to help make the process easier and more efficient. For instance, Spring Boot is used for the backend of the system. This is because it Spring Boot makes programming REST APIs incredibly simple with its several layers of abstraction it implements. In addition to this, it is powered by Java which is a programming language that is universally used and appreciated by developers, and thus making the project more accessible to contributors. On the other hand, D3.js, React, Material UI are used for the frontend, in addition to GraphQL. These APIs and frameworks are chosen for their simplicity and professional look. GraphQL supplements the crawling process to collect the necessary data from GitHub. Without this, the relational database would contain no data and migration would not be possible. React and Material UI go hand-in-hand for polished and high-quality websites. Moreover, these libraries give developers an immense array of tools for lots of different styles of project. Finally, D3.js is arguably the most important part of the project specification since it is the library that encompasses the visualisation aspect of the project. D3 is incredibly efficient to use, and yet so powerful in nature. It can support thousands of nodes whilst seamlessly reading a database from the backend in a matter of seconds.

To summarise, the following APIs, frameworks, and libraries are chosen to support the design and development of the project:

1. GraphQL
2. React
3. Material UI
4. D3.js
5. Spring Boot

1.1 Project Prototype Screenshot

Most final reports will contain a mixture of figures and charts along with the main body of text. The figure caption should appear directly after the figure as seen in Figure 1.1 whereas a table caption should appear directly above the table. Figures, charts, and tables should always be centered horizontally.



Figure 1.1: Logo of RHUL.

Chapter 2: Crawler

Crawling the necessary information was the very first task for the project. To do this, the GitHub GraphQL API was used. The code for this milestone was split up into three Python files. The first one was the main `crawl.py` file containing the code for making the API calls to the GitHub GraphQL API endpoint. The second one was the `logger.py` file to initialise the logger, so appropriate messages could be outputted to the console. The second and final one was the `split.py` file that split each individual crawled pull request into its own JSON file. The `logger.py` and `split.py` files were relatively straight forward to code and test, since they both performed simple functions. However, the `crawl.py` was a lot more complex in nature and, thus, was more problematic to write. This was the first problem encountered during this milestone.

In fact, there are two versions of this `crawl.py` file that have to be written during development. The first version is one that does not store the “cursor” value of the last pull request that it fetches, so it is not as efficient. A “cursor” value is effectively the unique identifier of each pull request as it is being fetched - it is NOT the ID of the pull request itself. The common issue with this first version of the crawler is that it uses up the 5,000 API calls per hour that a user gets using the GitHub GraphQL API. It also does this without retrieving the necessary amount of data (in this case, over 10,000 pull requests). As a result of this, the second version does store the “cursor” value of the last pull request that it fetches, and thus is a lot more efficient with its API calls. Only 1,000–2,000 API calls are required when using this second, improved version of the crawler, compared to all 5,000 API calls when using the original version.

The second problem encountered during this milestone was formatting and structuring the query string that would be sent as part of the callback to the GitHub GraphQL API endpoint. The format of this query string has to be very precise for the endpoint to accept it and not report back errors during compilation. After adjusting the formatting of this query string, the crawler does eventually function as expected.

Overall, the final, improved crawler file uses the `requests` pip library to make a callback to the GitHub GraphQL API, and uses the response from this callback to generate a collection of JSON files containing 10 pull requests per file. The crawler can accommodate for a single repository as input, as well as a dictionary of key-value pairs linking the owner of a repository with the repository’s name. As this crawl file is running, messages are outputted to the user in the console using the logger that was initialised earlier on. These files can then be read by the `split.py` file to make a JSON file for each pull request, ready for the next milestone of the project: Database.

Chapter 3: Database

The next stage of the project was designing a suitable database schema and then performing the migration process. This milestone was split into multiple parts: choosing the most suitable database solution, designing the database schema itself, and migration.

3.1 Database Solution

The first step in this milestone was choosing the most suitable database solution that was also most applicable to the use case of this project. Three candidates were considered: MySQL, PostgreSQL, and MariaDB. Early on, MySQL was thought to be obsolete since a lot of its features had been migrated into MariaDB and was now its subsidiary so that was eliminated immediately. The two remaining options, PostgreSQL and MariaDB, had their own advantages and disadvantages. In summary, PostgreSQL was supported by a vibrant community but also had known performance issues on large amounts of data (such as the pull request JSON information) and problems with compatibility since not all open-source applications support it. On the other hand, MariaDB had recently made MySQL a subsidiary which meant it already had an array of great features to offer. The first complaint made about MariaDB was surrounding the user interface, which would not be a problem for this project anyway since queries would mostly be done through the command line or through GUIs within IDEs. The second was negative performance impacts on large tables with multiple indices. However, this was not a major concern for development since the pull request data would only (ideally) need to be migrated once and then never again, so the performance impacts would not be very noticeable. Taking into account all the pros and cons, MariaDB was the ultimate choice for the project's database solution due to the overwhelming praise from the community and the plethora of features.

3.2 Database Schema

The second step was designing a database schema to store the pull request information that had been crawled in the previous stage. Initially, the idea was to have two tables: one for pull requests themselves, and another for the comments on those pull requests. However, this is not a complete solution since this schema can be further normalised to include individual author information in a separate table. As a result, a third table is introduced to help make database queries more efficient: the authors table. Furthermore, to allow the project to easily expand, a fourth table was implemented: the projects table. Each of these four tables contains indices for every non-timestamp column to allow for fast data retrieval. The final piece of the puzzle was taking advantage of foreign key attributes in every table to reduce the number of columns required. Regardless of the issues this may have caused later on, it was a worthy investment of the milestone since it made backend data retrieval a lot better later on. A full image of the complete schema in the form of an ER diagram is shown in the Figure 3.1.

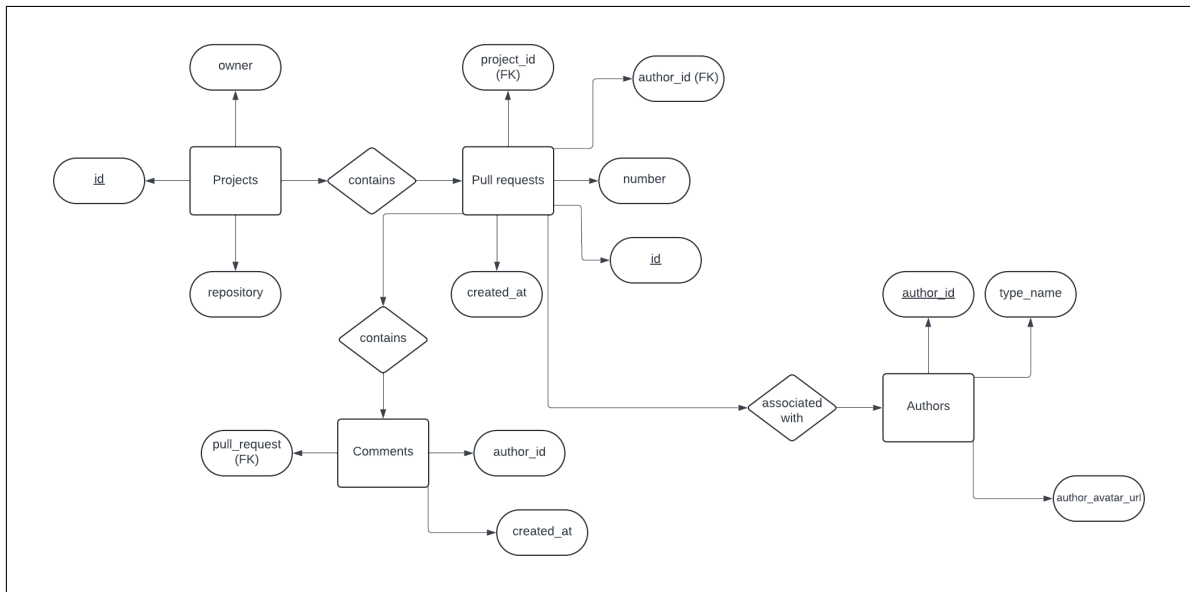


Figure 3.1: ER diagram of final database schema.

3.3 Migration

After the schema itself had been formed, the third and final step was storing the data. This, of course, could not be done manually due to the immense amount of data that was crawled (17,000+ pull requests, to be exact). The easiest way to automate this process was with a Python script that took advantage of the mysql-connector-python pip library. The biggest roadblock introduced when manipulating the contents of the schema was navigating the foreign key constraints. There were many exceptions that were thrown repeatedly throughout the development of this script because of these constraints. After much debugging, the best and most efficient solution was to partially add column values that did not rely on another table first before adding the rest. Another problem was deleted GitHub users, which caused problems since the author object inside the individual crawled JSON files did not exist. To counteract this issue, a “Ghost” user was added to the authors table during the instantiation of the database schema as a placeholder value for any deleted users. To avoid adding any duplicate users to the authors table, there was an internal list stored within the script that was compared against every time a pull request author was retrieved from the JSON data. This process was then repeated for each comment in every pull request to ensure no data redundancy/repetition. All these changes combined resulted in an efficient migration script that avoided data redundancy/repetition and foreign key constraint issues along the way.

Chapter 4: Frontend

The third stage of the project was the frontend. This was a vital stage is actually producing a functioning visualisation that would later be backed by the relational database content. Four sections were outlined for this stage: JavaScript vs TypeScript, ESLint & Prettier, developer relationship algorithm, & applying D3.js.

4.1 JavaScript vs TypeScript

The first choice that had to be made was what programming language should be used for the project. The two candidates for this were: JavaScript and TypeScript. Firstly, JavaScript was and will always be incredibly well-known in the frontend development community, making it the common go-to option. However, the lack of object-oriented programming concepts and tooling support makes it less enticing. For instance, being able to detect errors early on in a large project will be beneficial in the long-term, which is one of the many features that TypeScript has to offer over JavaScript. In contrast, TypeScript is purely statically typed which enhances error detection and, thus, ultimately, code quality. Additionally, TypeScript supports the latest ECMAScript features whilst also be a direct subsidiary of JavaScript, meaning that it supports all of its features by default. The primary downside to choosing TypeScript is the learning curve for amateur developers who are not used to typing statically. There are also extra compilation steps that are required to fully implement this static typing feature, therefore creating extra overhead as well. Regardless of this, TypeScript was ultimately chosen as the frontend programming language for this project due to its cutting-edge nature and abundance of features superior to JavaScript.

4.2 ESLint & Prettier

ESLint and Prettier were invaluable tools for the frontend development of this project. ESLint is a styling tool that, through static typing analysis, reports any compilation errors before the compilation process actually begins. For this project, the `airbnb` and `airbnb-typescript` ESLint configurations were used to ensure all the project code matched industry-level standards. All the necessary rules are stored inside a `.eslintrc.cjs` file.

The complete ESLint configuration file is shown below:

```
module.exports = {
  root: true,
  env: {browser: true, es2020: true},
  extends: [
    'eslint:recommended',
    'plugin:@typescript-eslint/recommended',
    'plugin:react-hooks/recommended',
    'airbnb',
    'airbnb-typescript',
    'plugin:react/jsx-runtime',
  ],
  ignorePatterns: ['dist', '.eslintrc.cjs', 'vite.config.ts'],
}
```

```

    parser: '@typescript-eslint/parser',
    parserOptions: {
      project: './tsconfig.json',
    },
    plugins: ['react-refresh'],
    rules: {
      'react-refresh/only-export-components': [
        'warn',
        {allowConstantExport: true},
      ],
      'linebreak-style': [
        'error',
        (process.platform === 'win32' ? 'windows' : 'unix')
      ]
    },
  },
}

```

Additional attributes such as `extends`, `parserOptions`, and `linebreak-style` are used to customise the configuration to work correctly with all the project's installed dependencies.

On top of that, Prettier is also used. This tool focuses more on the formatting of the project code rather than the styling of it. Thus, the readability of the code is more accessible and the project is easier to contribute to in industry. A bonus of using Prettier is that, unlike ESLint, it does not require a separate configuration file to function correctly. In fact, the project just used to default Prettier settings without a separate configuration being necessary.

4.3 Developer relationship algorithm

A crucial part of the frontend stage was establishing how the visualisation would associate development with their contributions to a project's pull requests. The visualisation worked by determining the size of a node (i.e. a developer) by how many comments they made on the individual project. The links between nodes (i.e. other developers) were created when one developer responded to another developer's comment on the SAME pull request. The thickness of these links between nodes (i.e. other developers) was determined by how many comments two developments made between each other on the SAME pull request.

With that context in mind, the relationship between developers was determined in the following way:

1. The system begins by initialising a list of pull requests, nodes, and links (to structure the graph).
2. All the pull requests (that are stored within the relational database) are stored in the pull request list.
3. For each pull request, if the author of the pull request is not already a node that exists in the graph (i.e. not a member of the nodes list), then add it with a size of zero.
4. For each comment in each pull request, if the author of the comment is not already a node that exists in the graph (i.e. not a member of the nodes list), then add it with a size of zero.

5. If they do exist, then increment the size of that node by one.
6. Store the author of the pull request and the author of the comment in a list, and sort this list.
7. If this pair of users (in the list) is not already in the links list, then add them with a thickness of zero.
8. Otherwise, increment the thickness between the pair of users by one.

4.4 Applying D3.js

The final step of the frontend milestone was to apply D3.js to have a visualisation that users can actually view. For the use case of the project, a force-directed graph was chosen as the appropriate D3 component. This was due to the easy ability to add nodes and links in a JSON format that could be easily retrieved from the backend API endpoint. An important point to mention is that the backend API endpoint was only callable thanks to the axios npm library. Axios allowed the API endpoint to be called and receive and store the response in a single variable, with a “data” attribute being used to retrieve it. The previously mentioned developer relationship algorithm was what provided the data compatible with the force-directed graph. In particular, this required a JSON response with “nodes” and “links” objects, respectively. These objects also required their own specific attributes, such as size for “nodes” and thickness for “links”. Several D3 forces and physics-related components were applied to make the visualisation successful due to the structure of the graph itself. With some sizes and thicknesses being inconsistent with each other (depending on the time period scale that was chosen), making sure the physics of the graph was smooth was important. It took some testing and trial-and-error to ensure that the physics were professional-looking but also practical, but eventually everything came into place. One major benefit to using D3 was how it seamlessly applied normalisation and scaling to quantitative scales. For instance, a separate function was required to normalise and scale the thickness of each link between nodes since at first they were too varying in size. This was needed in other numerical aspects of the graph as well. Another surprise when finally bringing the frontend together was how efficient the performance was when all the nodes were added to the graph. Despite load times reaching upwards of one minute, the graph component was stable enough to be usable despite thousands of nodes being present.

Chapter 5: Backend

The backend milestone was where everything became connected. The frontend visualisation linked to the backend API endpoint, labelled `/getGraph` (alongside the necessary request parameters), to show data that was actually relevant to the user, rather than placeholder mock data. Due to the gravity of this milestone, it is, of course, split into several stages: the power of Spring Boot, implementing the API endpoint itself, and (unit) testing.

5.1 The power of Spring Boot

Without Spring Boot, there would have been a lot more work in implementing the API endpoint. Spring Boot was incredibly powerful in terms of what was achieved with such little code. All that was needed was the instantiation of two list objects (two `ArrayList` objects, in particular) with particular attributes, and Spring Boot handled the rest of the conversion to a JSON object. Of course, adding specific values to these list objects was more challenging and complex as explained in the Frontend chapter. A single controller class controlled the entire endpoint, and there was no concern about this since there was no sensitive data being transferred other than publicly available information and only a single endpoint was being developed for the system. Hence, request parameters, within the URL of the endpoint itself, were used to pass the owner of the project and the project name. A start and end timestamp were optional parameters, implemented through two Material UI Date Time Pickers in the frontend system. From the use of Java functions to perform endpoint actions, this made testing very straightforward later on.

5.2 Implementing the API endpoint itself

Implementing the actual API endpoint itself was easier said than done, primarily due to the complex nature of the algorithm powering the relationship between developers. For the algorithm to be implemented properly, three new objects had to be created in the backend: Node, Link, and Graph. The Node class stored author information for every pull request that was stored in the pull requests table in the relational database alongside a numerical size value. This numerical size value was determined based on the number of comments an author has made on the whole project. The Link class stored the associated developers who communicated through a comment(s) on the SAME pull request, containing a source and target node alongside a numerical thickness value. This numerical thickness value is based on the number of comments made between two developers on the SAME pull request(s). Finally, the Graph class stored a list of Nodes and Links respectively in the form of two `ArrayList` objects. This Graph class is what was actually returned by the API endpoint, resulting in the carefully structured JSON response.

5.3 Testing

Testing was of lower priority when developing the backend system of the project, however, it was still an important aspect. Regardless of whether users only viewed the visualisation,

it was still paramount that all tests passed with adequate test coverage. It was discovered early on in the testing process that it would be inefficient and unwise to perform tests based on live production database information. This was because the database information could be removed/changed at any time. Subsequently, an in-memory database solution, H2 in particular, was applied for this purpose. The primary reason why was chosen was for that very reason: H2 is in-memory only. This means that the contents of the database are wiped between test runs, and thus helps maintain a consistent testing environment. It also means that the contents of the tests can be manually chosen and not reliant on live production data. In the end, all tests passed successfully, with twenty eight tests being run in total, with 97% overall test coverage.

Chapter 6: **Reflection**

Test